KONZEPT FÜR EINEN PROOF OF CONCEPT · OKTOBER 2026

Ariadne

Ein Lernagent auf Basis von Claude Opus 5.5, der den Weg kennt und ihn trotzdem nicht verrät.

Wie Menschen lernen, was daraus für einen KI-Tutor folgt, wie er gebaut ist, warum er plausibel funktioniert und wie man ihn einrichtet.



Niccolò Bambini: Ariadne und Theseus

Zielgruppe: Studierende der Informatik im höheren Semester
Modelle: Claude Opus 5.5 (Tutor) · Haiku 5.5 (Prüfer) · Sonnet 5 (Eskalation)
Plattform: Claude Code, lokal, alles als Klartext im Git-Repo

Concept & Architecture: Julian Trude
Authored with Claude Opus 5.5

Inhalt

1	Kurzfassung	
2	Der Einwand zuerst: KI kann Lernen schaden	Bastani, Kestin und drei Einwände aus der Zielgruppe
3	Wie Menschen lernen	elf Mechanismen mit Belegen
4	Was daraus für einen KI-Tutor folgt	das Labyrinth und zwölf Designprinzipien
5	Architektur	Harness, Lernermodell, Hinweisleiter, Quellenprüfung
6	Ein Lerntag mit Ariadne	Klausurmodus und Forschungsmodus, durchgespielt
7	Warum das plausibel funktioniert und wo die Grenzen liegen	
8	Einrichtung, Bau und Test	
9	Verworfene Alternativen	
A	Quellen	

WIE DIESES DOKUMENT ARGUMENTIERT

Ariadne ist nicht empirisch untersucht. Das Dokument behauptet deshalb nicht, dass sie wirkt. Es begründet, **warum es plausibel ist**, dass sie wirkt: Jedes Designelement wird auf einen Lernmechanismus zurückgeführt, und jeder Mechanismus auf veröffentlichte Studien. Wo die Belege dünn sind oder nur eingeschränkt übertragbar, steht das dabei.

Die Belegstärke ist durchgehend markiert: HOCH heißt vielfach repliziert oder durch Metaanalysen gestützt. MITTEL heißt durch einzelne gute Studien gestützt. NIEDRIG heißt plausibel abgeleitet, aber nicht direkt untersucht.

1 Kurzfassung

Generative KI kann Lernen messbar verschlechtern. Sie kann es aber auch messbar verbessern. Welcher Fall eintritt, entscheidet nicht das Modell, sondern das Design drumherum. Ariadne ist ein Entwurf, der konsequent auf der richtigen Seite dieser Linie gebaut ist.

Der Ausgangsbefund. In einem Feldexperiment mit knapp tausend Schülern verbesserte ungebremster GPT-4-Zugang die Übungsnoten um 48 %. In der Prüfung danach, ohne KI, schnitten dieselben Schüler aber 17 % schlechter ab als Schüler, die nie KI hatten [1]. Eine Variante mit didaktischen Leitplanken hob diesen Schaden weitgehend auf. Ein anderer, didaktisch gebauter KI-Tutor brachte Harvard-Studierenden in weniger Zeit mehr als den doppelten Lernzuwachs eines guten aktiven Präsenzunterrichts [2].

Das Prinzip. Ariadne kennt den Weg durchs Labyrinth, verrät ihn aber nicht. Zu jeder Aufgabe erstellt sie vorab eine versteckte *Lösungslandkarte*: Schlüsseleinsichten, typische Fehlvorstellungen, Zeitfallen. Der Lernende muss jeden Schritt zuerst selbst versuchen. Erst dann hilft Ariadne, und zwar über eine Hinweisleiter in vier Stufen, von der Rückfrage bis zur vollständigen Erklärung. Lehrreiche Sackgassen lässt sie zu, unproduktive verhindert sie. Wo immer möglich, widerlegt nicht die KI einen Irrtum, sondern die Maschine: Vorhersage aufschreiben, Code ausführen, vergleichen.

Die Technik. Ariadne läuft in Claude Code, lokal, mit dem Abo. Alles ist Klartext in einem Git-Repo: Regeln, Lernermodell, Sitzungsprotokolle. Was zwingend gelten muss, setzen *Hooks* in Code durch, nicht der Prompt:

- Jedes Zitat wird gegen den Text der angegebenen Seite geprüft. Ein erfundenes Zitat kommt nicht durch.
- Jede fachliche Antwort prüft zusätzlich ein günstiges Zweitmodell (Haiku 5.5).
- Das Lernermodell kann der Tutor nur über ein Skript mit festen Regeln ändern.
- Wiederholungen plant der FSRS-Algorithmus.

Zwei Modi. Im *Klausurmodus* wird Kursstoff erarbeitet, wiederholt und unter Prüfungsbedingungen geübt. Im *Forschungsmodus* werden Paper seziert: Fragestellung, Design, Statistik, Schwächen der Aussagekraft. Literatursuche gehört ausdrücklich nicht dazu; dafür sind Google Scholar und Zotero besser, schneller und sparsamer.

Warum das plausibel funktioniert. Ariadne erfindet kein neues Lernen. Sie macht die Techniken billig, deren Wirksamkeit am besten belegt ist, die Studierende aber kaum nutzen, weil sie mühsam sind: Abrufübung, verteiltes Wiederholen, Selbsterklärung, sofortiges Feedback, verschachteltes Üben [3]. Zeit spart sie vor allem an vier Stellen:

- weniger unproduktives Feststecken,
- Feedback sofort statt erst in der nächsten Übung,
- kein Wiederlesen, das sich nach Lernen anfühlt, aber keins ist,
- Wiederholung genau dann, wenn das Vergessen einsetzt.

Was offen bleibt. Die Kernstudien stammen aus Schulmathematik und Einführungsphysik und messen überwiegend kurzfristig. Die Übertragung auf Informatik im höheren Semester ist plausibel, aber nicht belegt. Die Quellenprüfung garantiert, dass ein Zitat existiert, nicht aber, dass es die Aussage stützt. Kapitel 7 benennt diese Grenzen einzeln.

2 Der Einwand zuerst: KI kann Lernen schaden

Wer gegenüber KI im Studium skeptisch ist, hat dafür einen guten empirischen Grund. Dieses Kapitel stellt ihn an den Anfang, weil das gesamte Konzept als Antwort darauf gebaut ist.

2.1 Bastani et al. (2025): Die Krücke

Bastani, Bastani, Sungu und Kollegen ließen knapp tausend Mathe-Schüler in der Türkei während regulärer Übungsstunden mit einem von zwei GPT-4-Tutoren arbeiten. Die Studie erschien 2025 in den *Proceedings of the National Academy of Sciences* [1]. Die beiden Varianten:

- **GPT Base** verhielt sich wie die normale ChatGPT-Oberfläche.
- **GPT Tutor** hatte Prompts, die das Lernen schützen sollten: Er gab Hinweise statt Lösungen.

Danach schrieben alle eine Prüfung **ohne** KI.

Gruppe	Übung mit KI	Prüfung ohne KI, verglichen mit Gruppe ohne KI
GPT Base	+48 %	**−17 %**
GPT Tutor	+127 %	Schaden „largely mitigated“, also weitgehend aufgehoben

Die Autoren deuten den Befund so: Ohne Leitplanken benutzen Schüler die KI als **Krücke**. Sie lassen sich die Lösung geben, statt sie zu erarbeiten. Das sieht in der Übung nach Leistung aus. Gelernt wird dabei aber nichts, und das zeigt sich, sobald die Krücke weg ist.

> **ZUM ERRATUM**
>
> Zu der Studie gibt es eine Korrektur (PNAS 122(34), e2518204122). Sie betrifft ausschließlich eine Autorenzugehörigkeit. Methodik und Befunde sind unverändert.

Was die Studie zeigt, und was nicht. Sie zeigt nicht, dass KI dem Lernen schadet. Sie zeigt, dass *dasselbe Modell* je nach Gestaltung schadet oder nicht. Beide Gruppen nutzten GPT-4. Den Unterschied machten allein die Anweisungen, wie der Tutor helfen durfte. Für jede Diskussion über KI im Studium ist das der wichtigste Punkt: Die Frage lautet nicht „KI ja oder nein“, sondern „welches Design“.

2.2 Kestin et al. (2025): Die Gegenprobe

Kestin, Miller und Kollegen verglichen in einem Harvard-Physikkurs mit 194 Studierenden zwei Formen des Lernens [2]:

- einen KI-Tutor, gebaut nach denselben didaktischen Prinzipien wie der Präsenzunterricht,
- **aktiven** Präsenzunterricht mit Peer Instruction und Kleingruppenarbeit. Die Vergleichsgruppe bekam also keine Frontalvorlesung, sondern gute Lehre.

Das Design war ein Crossover: Jede Person durchlief beide Bedingungen, in zwei aufeinanderfolgenden Wochen mit zwei Themen.

Messgröße	Ergebnis
Lernzuwachs gegenüber dem Vortest	beim KI-Tutor im Median mehr als doppelt so hoch
Effektstärke	0,63 laut linearer Regression; 0,73 bis 1,3 Standardabweichungen mit Korrektur des Deckeneffekts (Quantilsregression)
Zeitaufwand	Median 49 Minuten mit KI gegenüber 60 Minuten im Präsenzunterricht

Für das Konzept hier sind zwei Details aus der Studie wichtiger als die Schlagzahl:

1. **Ein Systemprompt allein reichte nicht.** Die Autoren berichten, dass er die Struktur mehrteiliger Aufgaben nicht zuverlässig hielt; der Tutor sprang zwischen Teilaufgaben. Deshalb erzwang die Plattform die Reihenfolge. Daraus folgt Designprinzip P11: Struktur gehört in Code, nicht in den Prompt.
2. **Halluzinationen wurden durch vorab geschriebene Lösungen eingedämmt.** Der Tutor bekam ausgearbeitete Musterlösungen Schritt für Schritt in den Prompt. Daraus folgt die Lösungslandkarte (P3).

GRENZE DER STUDIE

Kestin misst nur direkt nach der Lektion. Ob der Vorsprung nach Wochen noch besteht, ist nicht untersucht. Bastani zeigt aber gerade, dass der Unterschied zwischen kurzfristiger Leistung und echtem Lernen entscheidend sein kann. Die Autoren selbst nennen verteiltes Wiederholen als nächsten Schritt. Ariadne baut es fest ein (Abschnitt 5.4).

2.3 Drei Einwände aus der Zielgruppe

Einwand 1: „Zusammenhänge muss ich selbst finden. Wenn sie mir jemand präsentiert, lerne ich sie nur auswendig.“

Darin steckt viel Richtiges. Was man selbst hervorbringt, behält man besser als das, was man nur liest. Das ist der Generierungseffekt [4]. Anstrengung beim Lernen ist oft ein Zeichen, dass gelernt wird, nicht ein Hindernis; Bjork nennt das *desirable difficulties* [5]. Bastani zeigt genau den befürchteten Schaden, wenn die KI präsentiert statt fordert. Ariadne ist deshalb auf der Seite dieses Einwands gebaut: Sie präsentiert keine Zusammenhänge zum Mitschreiben.

Wo die Forschung widerspricht:

- **Selbstfinden ganz ohne Anleitung ist für Neulinge in einem Gebiet nachweislich weniger wirksam** als angeleitetes Lernen [6, 7]. Wer keine Struktur im Kopf hat, sucht im Dunkeln und überlastet das Arbeitsgedächtnis (Abschnitt 3.1).
- **Stundenlanges Feststecken an einer irrelevanten Stelle ist kein produktives Ringen,** sondern verlorene Zeit.
- **Selbst gefundene Zusammenhänge sind oft falsch, ohne dass man es merkt.** Menschen halten ihr eigenes Verständnis systematisch für tiefer, als es ist [8].

Die Antwort ist deshalb nicht „Ariadne findet die Zusammenhänge für dich“. Sie lautet: **Du findest sie selbst. Ariadne übernimmt die drei Dinge, die man allein schlecht kann.** Sie stellt die richtige Frage im richtigen Moment, sie meldet sofort zurück, ob der gefundene Zusammenhang trägt, und sie deckt Fehlvorstellungen auf, die man selbst nicht bemerken würde.

Einwand 2: „Für Recherche bin ich mit Google Scholar schneller und energetisch günstiger."

Das stimmt, und Ariadne recherchiert deshalb nicht. Sprachmodelle erfinden Literatur: In einer Untersuchung von 636 Literaturangaben aus KI-generierten Übersichten waren bei GPT-3.5 55 % und bei GPT-4 noch 18 % frei erfunden [9]. Neuere Modelle sind besser, aber eine Suchmaschine über einen echten Index ist für das Finden von Literatur grundsätzlich das richtige Werkzeug.

Ariadnes Forschungsmodus beginnt **erst, wenn das Paper gefunden ist**, nämlich bei der Frage, ob man es verstanden hat. Was ist die Hypothese? Misst das Design, was es zu messen behauptet? Was bedeutet diese Effektstärke? Welche Schwäche der Aussagekraft hat das Paper selbst nicht genannt? Das kann keine Suchmaschine beantworten.

Der Energieeinwand ist berechtigt und wird in Abschnitt 7.5 mit Zahlen behandelt.

Einwand 3: „Die KI halluziniert. Ich kann ihr nichts glauben."

Auch das ist berechtigt, und „das Modell ist inzwischen gut genug" wäre keine Antwort. Ariadne geht deshalb einen anderen Weg: Man soll ihr nicht glauben müssen. Fachliche Aussagen tragen eine Belegklasse. Zitate werden in Code gegen die Quelle geprüft, und was sich ausführen lässt, wird ausgeführt statt behauptet. Abschnitt 5.6 beschreibt, was dabei garantiert ist und was nicht.

3 Wie Menschen lernen

Dieses Kapitel fasst zusammen, was die kognitive Psychologie über Lernen weiß, soweit es für den Entwurf eines Tutors relevant ist. Es ist kein Lehrbuch, sondern das Fundament, auf das jedes Designelement in Kapitel 4 verweist.

3.1 Das Nadelöhr: Das Arbeitsgedächtnis

Alles, was gelernt wird, muss durch das Arbeitsgedächtnis. Dieses kann nur wenige Informationseinheiten gleichzeitig halten; die Schätzungen liegen bei etwa vier [10]. Die **Cognitive Load Theory** leitet daraus ab: Lernen scheitert oft nicht an mangelnder Intelligenz, sondern daran, dass zu viele unverbundene Elemente gleichzeitig verarbeitet werden müssen [11]. Wer eine Aufgabe ohne jedes Vorwissen durch blindes Probieren löst, verbraucht die Kapazität für die Suche und hat keine mehr für das Lernen.

HOCH *Folge für den Tutor:* Er dosiert, wie viel gleichzeitig neu ist. Hat jemand noch gar kein Vorwissen, braucht er Struktur, keine freie Suche.

3.2 Verständnis heißt: vernetzte Schemata

Das Langzeitgedächtnis ist praktisch unbegrenzt. Es speichert Wissen nicht als Liste, sondern als **Schemata**: vernetzte Strukturen, die viele Einzelheiten zu einer Einheit bündeln. Eine Expertin sieht in einer Seitentabelle nicht zwölf Felder, sondern ein einziges Konzept. Damit bleibt ihr Arbeitsgedächtnis für das eigentliche Problem frei. „Verstehen“ im Unterschied zu „Auswendiglernen“ bedeutet in diesem Bild: Neues ist an vorhandene Schemata angeschlossen, sodass man daraus ableiten kann, statt nur abzurufen.

HOCH *Folge:* Der Tutor muss wissen, welche Schemata schon da sind. Das leistet das Lernermodell. Er muss Verbindungen herstellen lassen, nicht vorsagen.

3.3 Abrufen stärkt das Gedächtnis mehr als Wiederlesen

Einer der robustesten Befunde der Lernforschung ist der **Testing-Effekt**. Wer Stoff aus dem Gedächtnis abruft, behält ihn langfristig deutlich besser als jemand, der ihn in derselben Zeit wiederholt liest [12]. Das gilt sogar für **erfolglose** Abrufversuche: Wer vor dem Lernen eine Frage zu beantworten versucht und scheitert, lernt die Antwort danach besser [13, 14]. Der Versuch bereitet das Gedächtnis darauf vor, die Lösung einzuordnen.

HOCH *Folge:* Erst versuchen, dann Hilfe bekommen. Das ist der Kern der Hinweisleiter.

3.4 Verteilen schlägt Pauken

Dieselbe Lernzeit, auf mehrere Termine verteilt, führt zu deutlich besserem Langzeitbehalten als geballtes Lernen. Eine Metaanalyse über Hunderte Experimente bestätigt das [15]. Am wirksamsten ist der Abstand, wenn die Wiederholung kurz vor dem Vergessen liegt. Spaced-Repetition-Algorithmen wie FSRS schätzen diesen Zeitpunkt pro Lerninhalt aus der bisherigen Abrufgeschichte [16].

HOCH *Folge:* Ein eingebauter Wiederholungsplaner, nicht als Option, sondern als Pflicht.

3.5 Generieren und erwünschte Schwierigkeiten

Wer eine Information selbst erzeugt, etwa ergänzt, herleitet oder vorhersagt, behält sie besser als jemand, der sie nur liest [4]. Allgemeiner: Bedingungen, die das Lernen *im Moment* erschweren, verbessern oft das *langfristige* Behalten und den Transfer [5]. Das gilt für Abrufen statt Lesen, Verteilen statt Pauken und Mischen statt Blocken. Die Schwierigkeit muss allerdings lösbar sein. Eine unlösbare Aufgabe ist nicht erwünscht schwer, sondern nur frustrierend.

HOCH *Folge:* Der Tutor macht es nicht bequem. Er macht es so schwer, dass es gerade noch lösbar ist.

3.6 Selbsterklärung

Lernende, die sich beim Durcharbeiten eines Beispiels jeden Schritt selbst erklären („warum folgt das?“), verstehen die Prinzipien besser und lösen neue Aufgaben erfolgreicher [17]. Die Selbsterklärung zwingt dazu, Lücken im eigenen Verständnis zu bemerken und zu füllen.

HOCH *Folge:* Nach jeder gelösten Schlüsselstelle fordert der Tutor eine Erklärung in eigenen Worten.

3.7 Vorhersagefehler und Feedback

Feedback gehört zu den wirksamsten Einflüssen auf Lernen überhaupt, aber nicht jedes Feedback hilft [18]. Wirksam ist Feedback, das sich auf die Aufgabe und den Lösungsweg bezieht, zeitnah kommt und eine Lücke zwischen dem Ist und dem Ziel sichtbar macht. Feedback, das nur lobt oder die Person bewertet, hilft wenig. Besonders lernwirksam ist ein erlebter **Vorhersagefehler**: Wer vorher festgelegt hat, was passieren wird, und dann etwas anderes sieht, kann den Irrtum nicht mehr wegerklären.

HOCH *Folge:* Vorhersage aufschreiben, dann ausführen. Bei Low-Level-Themen ist die Maschine der überzeugendste Feedbackgeber.

3.8 Mischen statt Blocken

Wer verschiedene Aufgabentypen gemischt übt statt blockweise, schneidet in Tests besser ab, obwohl sich das Mischen beim Üben schwerer und weniger erfolgreich anfühlt [19]. Beim Mischen muss man jedes Mal erst erkennen, *welche* Methode passt. Genau das verlangt eine Klausur.

MITTEL *Folge:* Die Klausursimulation mischt Themen.

3.9 Die Kompetenzillusion

Menschen sind schlechte Richter ihres eigenen Lernens. Sie halten ihr Verständnis für tiefer, als es ist [8]. Sie bevorzugen Lernmethoden, die sich flüssig anfühlen, etwa Wiederlesen und Markieren, obwohl diese wenig bringen [3, 20]. In einer Physik-Studie lernten Studierende im aktiven Unterricht nachweislich mehr, **fühlten** sich aber, als hätten sie weniger gelernt als in der Vorlesung [21]. Für jede Form von Lernunterstützung ist das brisant: Was sich gut anfühlt, ist oft das Falsche. Ein bequemer KI-Tutor wäre deshalb gefährlich, und Bastanis GPT Base ist genau dieser Fall.

HOCH *Folge:* Der Tutor macht die Kalibrierung sichtbar, also den Abstand zwischen „ich glaube, ich kann das“ und „ich kann das“.

3.10 Anleitung, Produktives Scheitern und der Expertise-Reversal-Effekt

Zwei Befundlinien scheinen sich zu widersprechen, ergänzen sich aber:

- **Anleitung hilft Neulingen.** Ausgearbeitete Beispiele sind für Anfänger wirksamer als freies Problemlösen [6, 7]. Mit wachsendem Vorwissen kehrt sich das um: Was Anfängern hilft, bremst Fortgeschrittene. Das ist der *Expertise-Reversal-Effekt* [6].
- **Produktives Scheitern hilft beim Verstehen.** Wer sich zuerst an einem Problem versucht, dabei scheitert und *danach* die Erklärung bekommt, versteht das Konzept tiefer als jemand, der zuerst erklärt bekommt [22]. Eine Metaanalyse bestätigt das für konzeptuelles Wissen, vor allem wenn die anschließende Erklärung an die gescheiterten Versuche anknüpft [23].

Die Auflösung: Gescheiterte Versuche sind wertvoll, **wenn danach Anleitung folgt**. Wertlos ist es, die ganze Zeit ohne Anleitung zu suchen. Und wie viel Struktur jemand braucht, hängt vom Vorwissen ab.

HOCH für Anleitung bei Neulingen, MITTEL für Produktives Scheitern. *Folge:* Lehrreiche Sackgassen zulassen, aber auffangen. Bei fehlendem Vorwissen mit einem Beispiel starten.

3.11 Was Einzeltutoring so wirksam macht

Bloom beschrieb 1984 das „2-Sigma-Problem“: Schüler mit Einzeltutor schnitten um zwei Standardabweichungen besser ab als im Klassenunterricht [24]. Spätere Analysen fanden kleinere, aber immer noch große Effekte. VanLehn verglich 2011 systematisch: Menschliches Tutoring erreicht etwa 0,79 Standardabweichungen. Computer-Tutoren, die Schritt für Schritt begleiten, kamen mit etwa 0,76 fast heran [25].

Was gutes Tutoring ausmacht, beschrieben Wood, Bruner und Ross schon 1976 als **Scaffolding**, also Lernen am Gerüst [26]. Der Tutor übernimmt nur die Teile, die der Lernende noch nicht allein schafft, und nimmt die Unterstützung zurück, sobald er sie nicht mehr braucht. Daraus ergibt sich die **Kontingenzregel**: nach einem Fehlschlag mehr Hilfe, nach einem Erfolg weniger.

HOCH *Folge:* Die Hinweisleiter folgt der Kontingenzregel.

3.12 Übersicht

Mechanismus	Kernaussage	Beleg-stärke	Quelle
Arbeitsgedächtnis	Kapazität eng; zu viel Neues gleichzeitig verhindert Lernen	HOCH	10, 11
Schemata	Verständnis = Anschluss an vorhandene Strukturen	HOCH	11
Testing-Effekt	Abrufen schlägt Wiederlesen, auch erfolgloses Abrufen hilft	HOCH	12–14
Verteiltes Lernen	Abstände schlagen Pauken	HOCH	15, 16
Generierung	Selbst Erzeugtes bleibt besser	HOCH	4, 5
Selbsterklärung	Schritte selbst begründen deckt Lücken auf	HOCH	17
Feedback	aufgabenbezogen und zeitnah wirkt; Vorhersagefehler wirken stark	HOCH	18
Verschachteln	Mischen schlägt Blocken bei Transfer und Prüfung	MITTEL	19
Kompetenzillusion	Gefühltes Lernen ≠ tatsächliches Lernen	HOCH	8, 20, 21
Expertise-Reversal	Neulinge brauchen Struktur, Fortgeschrittene Freiraum	HOCH	6, 7
Produktives Scheitern	Scheitern vor der Erklärung vertieft Verständnis	MITTEL	22, 23
Scaffolding	Hilfe nur so viel wie nötig, nach Erfolg zurücknehmen	HOCH	25, 26

4 Was daraus für einen KI-Tutor folgt

Ein Tutor, der mehr weiß als der Lernende, steht vor einem Paradox: Er kennt die Antwort, aber gerade die Antwort ist das, was er nicht geben darf. Ariadne löst das mit einem Bild und zwölf Prinzipien.

4.1 Das Labyrinth

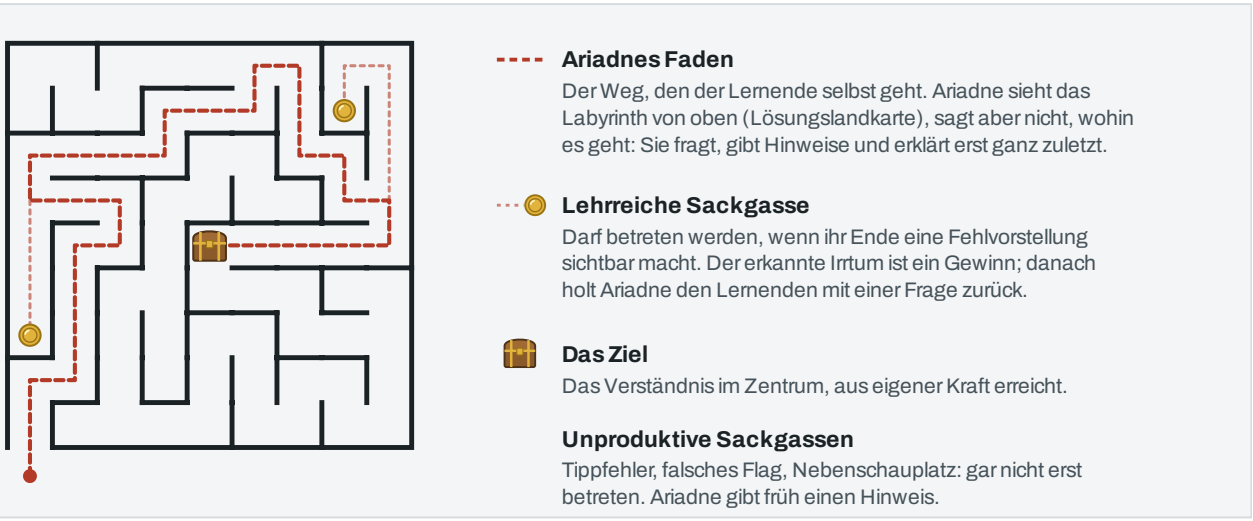


Abbildung 1: Das Leitbild. Ariadne kennt den Weg, verrät ihn aber nicht. Sie sorgt dafür, dass die eigene Anstrengung des Lernenden in die richtige Richtung geht.

Der Name stammt aus dem Mythos: Ariadne gibt Theseus einen Faden, der ihn sicher wieder hinausführt. Durch das Labyrinth gehen muss er selbst. Die Rollenteilung ist genau die, die die Forschung aus Kapitel 3 nahelegt:

- **Die Denkarbeit bleibt beim Lernenden:** Generieren, Abrufen, Selbsterklären.
- **Ariadne übernimmt Steuerung und Rückmeldung:** passende Aufgaben, Hinweise nach der Kontingenzregel, sofortiges Feedback, Wiederholungsplanung, Fehlvorstellungen sichtbar machen.

4.2 Zwölf Designprinzipien

#	Prinzip	Umsetzung in Ariadne	Gestützt auf
P1	**Erst versuchen, dann Hilfe**	Zu jeder Schlüsselstelle liefert der Lernende zuerst einen eigenen Versuch: Vorhersage, Erklärung oder Ansatz.	Testing- und Pretesting-Effekt [12–14], Bastani [1]
P2	**Hinweisleiter mit Kontingenz**	Vier Stufen: Rückfrage, Hinweis, Teillösung, vollständige Erklärung. Nach einem Fehlschlag eine Stufe höher, nach einem Erfolg zurück auf Stufe 1.	Scaffolding [25, 26]
P3	**Lösungslandkarte vorab**	Vor jeder Aufgabe erstellt Ariadne eine versteckte Karte: Schlüsselstellen, Fehlvorstellungen, Zeitfallen. Die Karte ist an das Kursmaterial gebunden und wird geprüft.	Kestin [2]

P4	**Lehrreiche Sackgassen zulassen**	Fehlvorstellungen, die in der Karte als lehrreich markiert sind, dürfen ausgelebt werden. Zeitfallen werden früh entschärft.	Produktives Scheitern [22, 23]
P5	**Die Maschine widerlegt**	Wo möglich: Vorhersage aufschreiben, ausführen (Code, `perf`, `gdb`, Python), vergleichen. Die Rückmeldung kommt von der Realität, nicht von der KI.	Feedback, Vorhersagefehler [18]
P6	**Selbsterklärung einfordern**	Nach jeder Schlüsselstelle: „Erklär in zwei Sätzen, warum das so ist." Ariadne prüft die Erklärung, nicht nur das Ergebnis.	Selbsterklärung [17]
P7	**Abrufen statt Wiederlesen, verteilt**	Jedes Konzept kommt in den FSRS-Planer. Wiederholt wird durch Abrufen, nie durch erneutes Lesen.	[12, 15, 16]
P8	**Mischen in der Prüfungssimulation**	Wiederholungen und Klausursimulation mischen Themen, damit man erkennen muss, welche Methode passt.	[19]
P9	**Kalibrierung sichtbar machen**	Vor jeder Antwort schätzt der Lernende seine Sicherheit von 1 bis 5. Die Bilanz zeigt Selbsteinschätzung gegen Trefferquote.	Kompetenzillusion [8, 20, 21]
P10	**Neulinge bekommen Beispiele**	Ohne Vorwissen im Lernermodell beginnt Ariadne mit einem ausgearbeiteten Beispiel und Selbsterklärungsfragen; mit wachsendem Vorwissen wird die Unterstützung zurückgenommen.	Expertise-Reversal [6, 7]
P11	**Struktur in Code, nicht im Prompt**	Regeln, die immer gelten müssen, setzen Hooks und Skripte durch: Quellenprüfung, Änderungen am Lernermodell, Wiederholungspflicht.	Kestin [2]
P12	**Quellenbindung mit Belegklassen**	Jede fachliche Aussage ist belegt (Seite), ausgeführt (Protokoll) oder als unbelegt markiert.	Kestin [2], Walters & Wilder [9]

4.3 Was Ariadne nie tut

HARTE GRENZEN

- Sie gibt keine Lösung, bevor ein eigener Versuch vorliegt. Die einzige Ausnahme ist der protokollierte Notausgang, siehe 5.3.
- Sie stellt keine Suggestivfragen. „Ist es nicht so, dass …?" ist eine verkappte Antwort. Leitfragen müssen den Denkschritt beim Lernenden lassen.
- Sie sucht keine Literatur und nennt keine Paper, die nicht im Material liegen.
- Sie bewertet nicht die Person, sondern den Lösungsweg.
- Sie schreibt keine Abgaben. Übungsblätter mit Bewertung, Hausarbeiten und Prüfungsleistungen sind tabu; geübt wird an Altklausuren und eigenen Aufgaben.
- Sie verändert das Lernermodell nicht nach eigenem Ermessen, sondern nur über feste Regeln, siehe 5.2.

5 Architektur

Ariadne ist kein eigenes Programm, sondern eine Konfiguration von Claude Code: ein Git-Repo mit Regeln, Skills, Hooks und ein paar kleinen Python-Skripten. Alles ist Klartext und damit lesbar, diffbar und änderbar.

5.1 Überblick

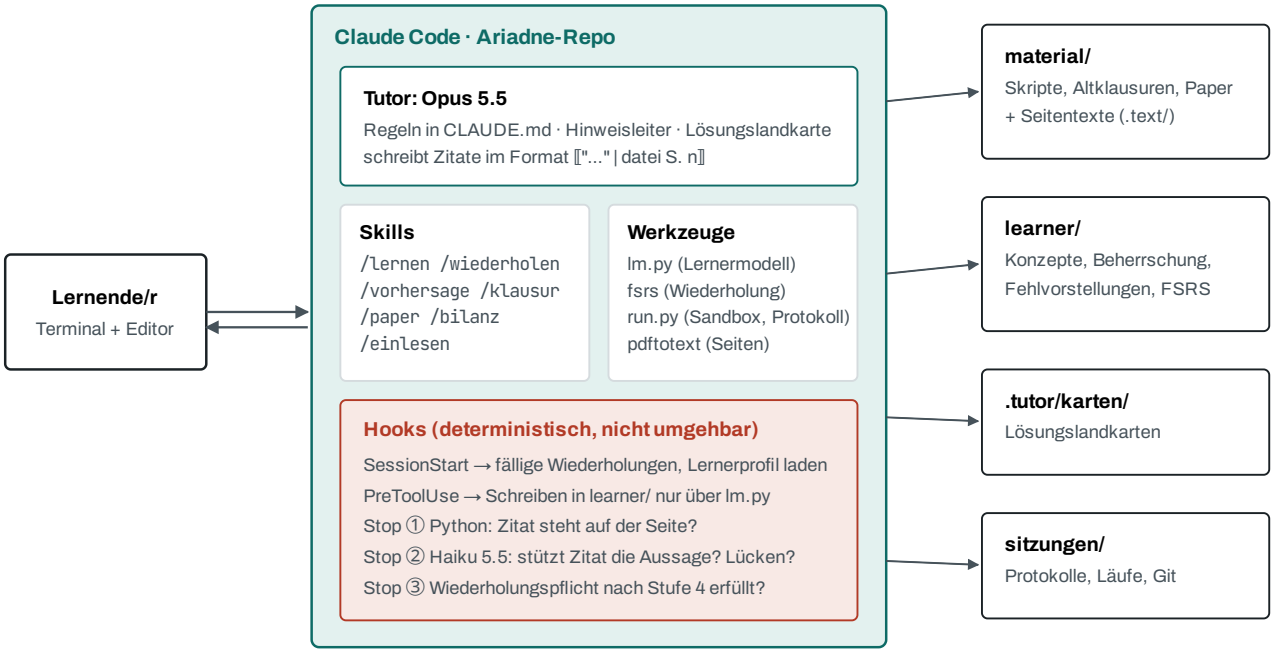


Abbildung 2: Bausteine. Der Tutor (Opus 5.5) arbeitet über Skills und Werkzeuge. Hooks laufen außerhalb des Modells und können Antworten blockieren. Alle Daten liegen als Dateien im Repo.

Repo-Aufbau

```
ariadne/
├── CLAUDE.md                  # Grundregeln des Tutors (Abschnitt 8.3)
├── .claude/
│   ├── settings.json          # Hook-Konfiguration
│   ├── skills/                # ein Ordner pro Lernmodus, je mit SKILL.md
│   │   ├── lernen/  wiederholen/  vorhersage/  klausur/
│   │   └── paper/   bilanz/  einlesen/  karte/
│   └── hooks/                 # session_start.py, guard_learner.py, stop_check.py
├── tools/
│   ├── lm.py                  # einziger Schreibweg ins Lernermodell
│   ├── review.py              # FSRS-Planer (Paket "fsrs")
│   ├── run.py                 # führt Code aus, protokolliert Ein- und Ausgabe
│   ├── ingest.py              # PDF → Seitentexte
│   └── pruefer_prompt.md      # Prompt für den Haiku-Prüfer
├── material/
│   ├── os/  skript.pdf  altklausur-2024.pdf  uebung-05.pdf
│   ├── paper/  bastani-2025.pdf
│   └── .text/os-skript/001.txt … 214.txt   # von ingest.py erzeugt
├── learner/
│   ├── konzepte.json          # Lernermodell
│   └── kalibrierung.csv
├── .tutor/karten/             # Lösungslandkarten, nicht für den Lernenden gedacht
├── sitzungen/2026-10-15/      # protokoll.jsonl, runs/
└── pruefer-validierung/       # Testpaare für den Haiku-Prüfer (8.5)
```

5.2 Das Lernermodell

Das Lernermodell ist eine JSON-Datei mit einem Eintrag pro Konzept. Konzepte werden beim Einlesen des Materials vorgeschlagen und vom Lernenden bestätigt. Ein Beispiel:

```
{
  "id": "os.vm.tlb",
  "titel": "Translation Lookaside Buffer",
  "quelle": "material/os/skript.pdf#S112",
  "voraussetzungen": ["os.vm.seitentabelle", "os.vm.adressuebersetzung"],
  "beherrschung": 2,                  // 0 unbekannt … 4 sicher, nur über Regeln geändert
  "fehlvorstellungen": [
    {"text": "TLB-Miss führt zu Page Fault", "gesehen": "2026-10-15", "behoben": true}
  ],
  "letzte_stufe": 3,                  // höchste Hinweisstufe in der letzten Sitzung
  "fsrs": {"stability": 3.1, "difficulty": 6.2, "due": "2026-10-18", "reps": 1, "lapses": 0},
  "notausgang": 0                     // wie oft "gib mir einfach die Lösung"
}
```

Warum das Modell das Lernermodell nicht frei bearbeiten darf. Ein Sprachmodell neigt dazu, Fortschritt freundlich zu bewerten. Ein Lernermodell, das sich selbst schönschreibt, würde die Kompetenzillusion aus 3.9 automatisieren. Deshalb blockiert ein `PreToolUse`-Hook jedes direkte Schreiben in `learner/`. Änderungen laufen nur über `tools/lm.py`. Das Skript leitet sie aus beobachtbaren Fakten ab, also aus der erreichten Hinweisstufe und dem Ergebnis des Abrufs, nicht aus dem Urteil des Modells:

Beobachtung in der Sitzung	FSRS-Bewertung	Beherrschung
Ohne Hinweis richtig, Selbsterklärung trägt	Good (Easy, wenn Sicherheit 5 und schnell)	+1
Richtig nach Stufe 1 bis 2	Hard	unverändert
Erst nach Stufe 3 oder 4, oder Notausgang	Again	–1, mindestens 0

5.3 Die Hinweisleiter

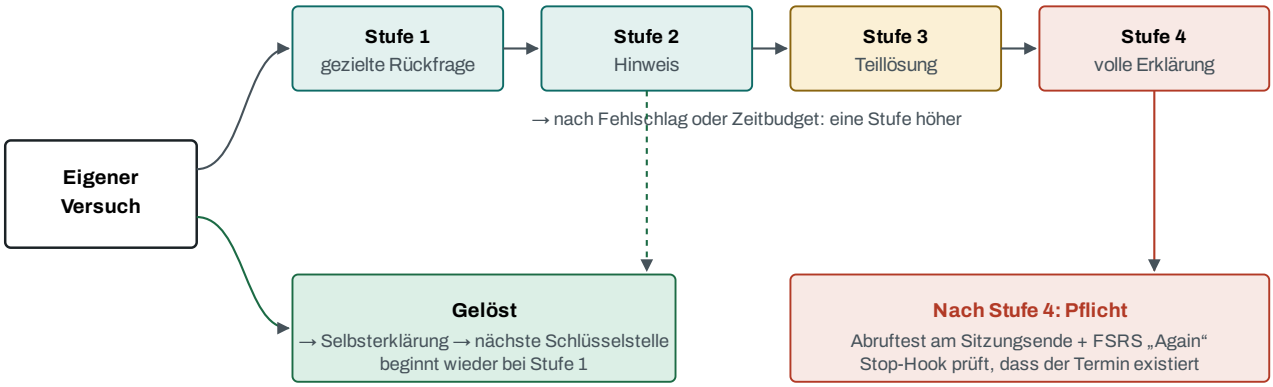


Abbildung 3: Hinweisleiter mit Kontingenzregel. Jede Schlüsselstelle beginnt mit einem eigenen Versuch. Hilfe steigt nur bei Bedarf und fällt nach jedem Erfolg wieder auf Stufe 1.

Stufe	Was Ariadne liefert	Beispiel (Thema TLB)
1 Rückfrage	Eine Frage, die auf den Fehler im Versuch zeigt, ohne ihn zu benennen	„Was genau steht in einem TLB-Eintrag?“
2 Hinweis	Richtung angeben, Schritt nicht vorwegnehmen	„Schau dir an, welche Information der TLB zwischenspeichert und welche nicht.“
3 Teillösung	Den ersten Teil des Schritts zeigen, den Rest offen lassen	„Der TLB speichert Übersetzungen. Was passiert also bei einem Miss, wenn die Seite im RAM liegt?“
4 Erklärung	Vollständig, mit Beleg; danach Selbsterklärung und Wiederholungspflicht	Erklärung mit Zitat aus dem Skript, anschließend „Erklär es in eigenen Worten.“

Zeitbudget. Jede Schlüsselstelle in der Lösungslandkarte hat ein Zeitbudget, typischerweise 5 bis 15 Minuten. Wird es überschritten, steigt Ariadne eine Stufe auf, auch ohne ausdrücklichen Fehlschlag. Das verhindert unproduktives Feststecken.

Notausgang. „Gib mir einfach die Lösung“ funktioniert immer. Der Lernende entscheidet frei. Der Notausgang wird aber protokolliert, zählt als Stufe 4 und taucht in der Bilanz auf. Ariadne verbietet nichts, aber sie macht die Kosten sichtbar.

5.4 Die Lösungslandkarte

Bevor eine Aufgabe beginnt, ruft Ariadne den Skill `/karte` auf und legt die Karte unter `.tutor/karten/` ab. Sie ist an das Material gebunden: Jede Schlüsselstelle verweist auf eine Seite und durchläuft dieselbe Quellenprüfung wie jede Antwort. Ein Beispiel:

```
aufgabe: "Warum ist Stride 4096 beim Speicherzugriff so viel teurer als Stride 64?"
konzepte: [os.vm.tlb, os.vm.seitentabelle, arch.cache.zeile]
schluesselstellen:
  - id: S1
    einsicht: "Der TLB speichert Übersetzungen virtuell→physisch pro Seite, keine Daten."
    beleg: ⟦"caches recent virtual-to-physical translations" | skript.pdf S. 112⟧
    zeitbudget_min: 8
  - id: S2
    einsicht: "Bei Stride 64 teilen sich 64 Zugriffe eine Seite, also einen TLB-Eintrag."
    zeitbudget_min: 6
  - id: S3
    einsicht: "Übersteigt die Zahl der berührten Seiten die TLB-Reichweite, verfehlt fast jeder Zu-
griff."
    zeitbudget_min: 10
fehlvorstellungen:
  - text: "TLB-Miss = Page Fault"
    lehrreich: true                       # zulassen, dann per perf widerlegen
    aufloesung: "perf stat -e dTLB-load-misses,page-faults zeigt Millionen Misses, kaum Faults"
  - text: "Der Cache allein erklärt den Unterschied"
    lehrreich: true
zeitfallen:
  - "Compiler optimiert die Schleife weg (-O2 ohne volatile)"    # früh Stufe-2-Hinweis
  - "perf fehlt die Berechtigung (perf_event_paranoid)"
```

Das Zitat in der Karte ist ein illustratives Beispiel. In einer echten Karte steht der Wortlaut des eingelesenen Skripts.

Die Karte ist nicht geheim im Sinne von Sicherheit. Wer sie liest, schadet nur sich selbst, und das Repo gehört dem Lernenden. Sie liegt in einem eigenen Ordner, damit man nicht versehentlich hineinschaut.

5.5 Modi und Skills

Skill	Modus	Was er tut
`/einlesen`	beide	Zerlegt neue PDFs in Seitentexte, schlägt Konzepte und Voraussetzungen vor; der Lernende bestätigt.
`/lernen`	Klausur	Erarbeitet ein Thema. Liest das Lernermodell und wählt das Einstiegsniveau: Beispiel bei Neulingen (P10), Problem bei Vorwissen. Erstellt die Lösungslandkarte, dann Schlüsselstelle für Schlüsselstelle nach der Hinweisleiter.
`/vorhersage`	Klausur	Für Low-Level-Themen: Code oder Experiment vorlegen, Vorhersage verlangen und festschreiben, dann über `run.py` ausführen und vergleichen.
`/wiederholen`	beide	Fragt fällige Konzepte aus dem FSRS-Planer gemischt ab. Immer zuerst Sicherheit von 1 bis 5 abfragen, dann Antwort, dann Bewertung über `lm.py`.
`/klausur`	Klausur	Simulation unter Zeitdruck, bevorzugt mit Altklausuraufgaben, Themen gemischt. Während der Simulation keine Hilfe; Auswertung danach.
`/paper`	Forschung	Seziert ein Paper in drei Durchgängen (unten). Der Lernende liefert in jedem Schritt zuerst seine Einschätzung.
`/bilanz`	beide	Wochenbilanz: Konzepte nach Beherrschung, Kalibrierung, Notausgänge, fällige Wiederholungen. Reine Auswertung der Dateien, ohne Urteil des Modells.

Der Forschungsmodus im Detail

Der Skill `/paper` folgt der Drei-Durchgänge-Methode von Keshav [27], erweitert um eine kritische Prüfung:

1. **Überblick (5 bis 10 Minuten).** Titel, Abstract, Einleitung, Überschriften, Fazit. Der Lernende beantwortet: Welche Art Paper? Welche Behauptung? Wie gehört es zum Kursstoff? Ist es das Weiterlesen wert?
2. **Inhalt.** Abbildungen, Tabellen und Methodik verstehen. Ariadne fragt nach jeder Abbildung: „Was zeigt die y-Achse, und was wäre das Ergebnis, wenn die Hypothese falsch wäre?“
3. **Kritische Prüfung.** Der Lernende arbeitet eine Checkliste ab. Ariadne gibt zu jedem Punkt erst Rückfragen, dann Hinweise:
   - Fragestellung und Hypothese, vorab festgelegt oder nachträglich?
   - Design: Randomisierung, Kontrollbedingung, Stichprobe, Ausfälle
   - Messung: Misst das Instrument, was behauptet wird?
   - Statistik: Effektstärken, Konfidenzintervalle, Mehrfachvergleiche, Deckeneffekte
   - Schwächen der Aussagekraft: intern und extern, also auf wen übertragbar?
   - Reproduzierbarkeit: Daten, Code, Vorregistrierung
   - Verbindung: Was bedeutet das für mein Thema oder meine Arbeit?

Ergebnis ist eine Zusammenfassung **in den Worten des Lernenden**, mit geprüften Zitaten. Ariadne korrigiert, schreibt sie aber nicht.

5.6 Quellenprüfung in drei Ebenen

Jede fachliche Aussage in Ariadnes Antworten trägt eine von drei Belegklassen:

Klasse	Bedeutung	Format in der Antwort
1 belegt	steht im Material, Wortlaut mit Seite	`⟦"wörtliches Zitat" | skript.pdf S. 112⟧`
2 ausgeführt	durch Ausführung geprüft	`⟦lauf: r3⟧`, verweist auf `sitzungen/…/runs/r3.json`
3 unbelegt	Modellwissen, nicht im Material	`⟦unbelegt⟧`, dazu ein ausdrücklicher Hinweis im Text

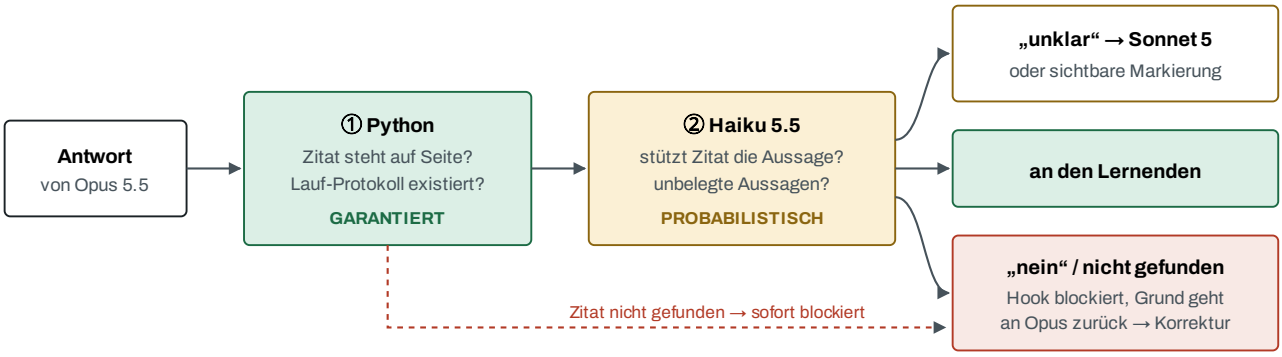


Abbildung 4: Prüfkette. Ebene ① ist reiner Code ohne Modell. Ebene ② ist ein günstiges Zweitmodell mit frischem Kontext. Nur Zweifelsfälle gehen an ein stärkeres Modell oder an den Menschen.

Ebene ①: Existenz, deterministisch

Beim Einlesen zerlegt `ingest.py` jedes PDF mit `pdftotext -layout` seitenweise in Textdateien. Nach jeder Antwort des Tutors läuft der Stop-Hook:

```
1. Letzte Antwort des Tutors aus dem Sitzungsprotokoll lesen.
2. Alle ⟦"…" | datei S. n⟧ per regulärem Ausdruck finden.
3. Je Zitat: Seitentext laden, beide Seiten normalisieren
   (Unicode NFKC, Silbentrennung am Zeilenende zusammenziehen,
    Leerraum vereinheitlichen, typografische Anführungszeichen angleichen),
   dann Teilstring-Suche.
   Zur Toleranz gegen Extraktionsfehler: auch Seite n-1/n+1 prüfen
   (Seitenumbrüche), aber Treffer dort als "falsche Seite" melden.
4. Seite leer oder fast ohne Text (Scan, Formelseite) → "nicht prüfbar",
   niemals "ok".
5. ⟦lauf: rX⟧ → existiert runs/rX.json, und stammt es aus dieser Sitzung?
6. Mindestens ein Fehler → {"decision": "block", "reason": "<Liste>"}
   Das Modell erhält die Liste und muss die Antwort korrigieren.
```

Was damit garantiert ist: Kein Zitat erreicht den Lernenden, das nicht wörtlich auf der angegebenen Seite steht, und kein Verweis auf einen Lauf, der nicht stattgefunden hat. Die einzigen Fehlerquellen sind Fehler der Textextraktion. Sie führen zu „nicht prüfbar“, nicht zu einem falschen „ok“.

Ebene ②: Stützung und Vollständigkeit, mit Haiku 5.5

Ist Ebene ① bestanden und enthält die Antwort fachliche Aussagen, ruft der Hook ein zweites Modell auf:

```
ARIADNE_PRUEFER=1 claude -p --model claude-haiku-5-5 \
  --output-format json "$(cat tools/pruefer_prompt.md) <antwort> <zitate mit Absatz>"
```

Der Prüfer sieht **nur** die Antwort und die zitierten Stellen mit ihrem umgebenden Absatz. Das Gespräch sieht er nicht. So kann ihn die Argumentation nicht mitziehen. Er liefert JSON:

```
{"aussagen": [
  {"text": "Ein TLB-Miss löst einen Page Walk aus, keinen Page Fault.",
   "beleg": "skript.pdf S. 112", "stuetzt": "ja"},
  {"text": "Moderne x86-CPUs haben einen zweistufigen TLB.",
   "beleg": null, "markiert_unbelegt": false, "stuetzt": "fehlt"}
 ],
 "gesamt": "nein"}
```

- `"nein"` bei mindestens einer nicht gestützten oder unmarkierten Aussage: Der Hook blockiert, das Modell korrigiert.
- `"unklar"`: Eskalation an Sonnet 5 mit derselben Eingabe. Sagt auch Sonnet 5 „unklar“, geht die Antwort mit einer sichtbaren Markierung `⚠ Beleg prüfen` durch.
- Ungültiges JSON oder Zeitüberschreitung: wird wie „unklar“ behandelt, nie wie „ja“.

Warum Haiku genügt. Es handelt sich nicht um eine offene Denkaufgabe, sondern um eine eng umrissene Prüfung mit kleinem Kontext: Folgt Aussage A aus Textstelle B? Welche Sätze sind Sachaussagen ohne Beleg? Der Prüfer-Prompt verlangt ausdrücklich, **im Zweifel „unklar“** zu sagen. Die Fehlerkosten sind ungleich: Ein Fehlalarm kostet zehn Sekunden Nachlesen, ein übersehener Fehler kostet falsches Wissen. Ob Haiku das zuverlässig genug kann, wird nicht angenommen, sondern gemessen (Abschnitt 8.5, Test T7).

Rekursion vermeiden. Der Prüfaufruf ist selbst eine Claude-Code-Sitzung und würde ohne Vorkehrung wieder die Hooks auslösen. Alle Hook-Skripte beenden sich deshalb sofort, wenn `ARIADNE_PRUEFER=1` gesetzt ist. Außerdem lässt der Stop-Hook höchstens zwei Korrekturrunden zu, erkennbar am Feld `stop_hook_active`. Danach geht die Antwort mit sichtbarer Warnung durch, statt endlos zu kreisen.

Was nicht garantiert ist

EHRLICHE GRENZE

Ob ein echtes Zitat die Aussage wirklich stützt, ist eine Bedeutungsfrage. Ob eine Aussage ohne Beleg und ohne Markierung durchrutscht, ebenfalls. Beides prüft ein Modell, also nie mit Sicherheit. **Hart garantiert ist nur Ebene ①.** Die letzte Instanz ist der Lernende: Weil jedes Zitat wörtlich mit Seite dasteht, ist die Prüfung in Sekunden selbst gemacht. Das ist gewollt, denn wer die Originalstelle liest, lernt dabei.

5.7 Modellaufteilung

Rolle	Modell	Wann	Begründung
Tutor	Claude Opus 5.5	jede Interaktion	Didaktik und Fachtiefe brauchen das stärkste Modell.
Prüfer	Claude Haiku 5.5	jede Antwort mit Fachaussagen	eng umrissene Prüfung, kleiner Kontext, ein Bruchteil des Verbrauchs
Eskalation	Claude Sonnet 5	nur bei „unklar“	selten, deshalb vertretbar
Existenzprüfung, Lernermodell, Planung	Python	immer	deterministisch, kostenlos, nachprüfbar

Ein zweiter Opus als Prüfer wurde verworfen. Er würde den Verbrauch etwa verdoppeln, ohne dass für eine eng umrissene Prüfaufgabe ein entsprechender Gewinn zu erwarten ist. Ob Haiku reicht, entscheidet die Validierung in 8.5, nicht eine Annahme.

5.8 Transparenz

Für eine skeptische Zielgruppe ist Transparenz kein Zusatz, sondern Voraussetzung. Ariadne ist so gebaut, dass man ihr nicht vertrauen muss, sondern nachsehen kann:

- **Jede Regel ist eine Datei.** Wer wissen will, warum Ariadne nicht vorsagt, liest `CLAUDE.md`. Wer es ändern will, ändert es und sieht den Unterschied im Diff.
- **Jede Sitzung ist protokolliert:** Dialog, Hinweisstufen, Läufe, Prüfergebnisse.
- **Jede Änderung am Lernermodell ist ein Git-Commit** mit Begründung. Der Fortschritt ist eine Historie, kein Gefühl.
- **Keine selbstständige Fortschreibung.** Ariadne ändert ihre eigenen Regeln und Skills nicht. Das unterscheidet sie bewusst von Agenten mit selbstlernendem Gedächtnis (Kapitel 9).

6 Ein Lerntag mit Ariadne

Ein durchgespieltes Beispiel, gekürzt. Die Dialoge sind konstruiert, um das Zusammenspiel der Bausteine zu zeigen; sie sind kein Protokoll einer echten Sitzung. Seitenzahlen und Zitate beziehen sich auf ein fiktives Betriebssystem-Skript.

6.1 Morgens: Wiederholung

HOOK	`SessionStart: 4 Konzepte fällig (os.sched.cfs, os.sync.mutex, os.vm.seitentabelle, stat.konfidenz). Kalibrierung letzte Woche: Sicherheit 4–5 bei 71 % Treffern.`
ARIADNE	Vier fällige Wiederholungen, gemischt. Erste Frage: Ein Prozess hat eine 4-stufige Seitentabelle auf x86-64. Wie viele Speicherzugriffe braucht eine Adressübersetzung ohne TLB-Treffer im ungünstigsten Fall? Wie sicher bist du, 1 bis 5?
LERNENDE	Sicherheit 4. Vier Zugriffe, einer pro Stufe.
ARIADNE	Richtig für die Übersetzung selbst. Wie viele Zugriffe sind es insgesamt, bis das Datum im Register steht?
LERNENDE	Fünf, plus der eigentliche Datenzugriff.
LM.PY	`os.vm.seitentabelle: richtig nach Stufe 1 → FSRS Hard, nächster Termin 2026-10-19`

> Die Bewertung ergibt sich aus der Hinweisstufe, nicht aus dem Urteil des Modells (5.2). Die Sicherheitsangabe geht in die Kalibrierung ein (P9).

6.2 Vormittags: Ein neues Thema erarbeiten

Thema: der TLB. Das Lernermodell kennt die Seitentabelle mit Beherrschung 3, den TLB nur mit 1. Es gibt also Vorwissen. Ariadne startet deshalb mit einem Problem, nicht mit einem Beispiel (P10), und erstellt zuerst die Lösungslandkarte aus 5.4.

ARIADNE	Hier ist ein kleines C-Programm. Es reserviert 1 GiB und berührt zuerst alles einmal. Danach misst es zwei Schleifen: Die erste liest jedes 64. Byte, die zweite jedes 4096. Byte, beide gleich oft. **Bevor du es ausführst:** Was zeigt `perf stat -e dTLB-load-misses,page-faults` für die beiden Schleifen? Schreib deine Vorhersage in `vorhersage.md`.
LERNENDE	Stride 4096 ist langsamer, weil jeder Zugriff eine neue Seite trifft. Also mehr TLB-Misses, und jeder TLB-Miss ist ein Page Fault, weil die Übersetzung nachgeladen werden muss. Ich erwarte bei 4096 also viele Page Faults.
KARTE	`Fehlvorstellung „TLB-Miss = Page Fault“ erkannt · lehrreich: true → nicht korrigieren, ausführen lassen`
ARIADNE	Vorhersage gespeichert. Führ es aus: `python tools/run.py stride.c`
RUN.PY R3	`stride=64: dTLB-load-misses 0,26 M · page-faults 3` `stride=4096: dTLB-load-misses 16,7 M · page-faults 2`
LERNENDE	Die Misses passen, aber die Page Faults sind praktisch null. Dann ist ein TLB-Miss offenbar kein Page Fault?

ARIADNE (Stufe 1) Genau den Widerspruch meine ich. Was muss die Hardware bei einem TLB-Miss tun, und wann braucht sie dafür das Betriebssystem?

LERNENDE Sie muss die Seitentabelle durchlaufen, das ist der Page Walk aus der Wiederholung heute früh. Das Betriebssystem braucht sie nur, wenn der Eintrag ungültig ist, also wenn die Seite nicht im RAM liegt. Weil wir vorher alles berührt haben, liegt alles im RAM.

ARIADNE Das trägt. Zur Kontrolle die Stelle im Skript: ⟦"a TLB miss is handled by walking the page table" | skript.pdf S. 113⟧ ✓. Erklär in zwei Sätzen, warum Stride 64 so wenige Misses hat.

STOP-HOOK `① Zitat auf S. 113 gefunden · Lauf r3 vorhanden · ② Haiku: 2 Aussagen, gestützt → durchgelassen`

> Das ist der Kern des Konzepts. Ariadne hat die Fehlvorstellung erkannt und bewusst nicht korrigiert. Widerlegt hat sie die Messung. Die entscheidende Einsicht kam vom Lernenden selbst und knüpfte an die Wiederholung vom Morgen an. Ariadne hat nur eine Frage gestellt (Stufe 1).

Wenn es nicht klappt

Wäre die Lernende nach der Rückfrage nicht weitergekommen, hätte Ariadne nach Ablauf des Zeitbudgets für S1 die Stufe erhöht. Stufe 2 wäre der Hinweis „Vergleich, welche Information der TLB zwischenspeichert, mit dem, was bei einem Page Fault fehlt“ gewesen. Wäre es bis Stufe 4 gegangen, hätte der Stop-Hook am Sitzungsende geprüft, dass `os.vm.tlb` mit „Again“ bewertet und für den nächsten Tag eingeplant ist.

Die Übertragungsfrage

ARIADNE Letzte Schlüsselstelle: Was würde sich bei Stride 4096 ändern, wenn der Speicher in 2-MiB-Seiten liegt? Erst vorhersagen, dann messen wir mit `madvise(MADV_HUGEPAGE)`.

6.3 Nachmittags: Ein Paper sezieren

Im Forschungsmodus liest die Lernende ausgerechnet das Paper, das diesem Konzept zugrunde liegt: Bastani et al. 2025 [1].

ARIADNE Durchgang 1 ist erledigt, deine Kurzfassung lautet: „KI schadet dem Lernen, Leitplanken helfen.“ Durchgang 3, Punkt Messung: Woran genau wurde der Schaden von 17 % gemessen, und unter welchen Bedingungen fand diese Messung statt?

LERNENDE An Prüfungsnoten. Die Prüfung war ohne KI, nach den Übungsstunden mit KI.

ARIADNE Und was folgt daraus für deine Kurzfassung? Sagt die Studie, dass KI dem Lernen schadet?

LERNENDE Nein, eher: Eine bestimmte Nutzung schadet. Beide Gruppen hatten dasselbe Modell, der Unterschied lag nur im Design. Meine Kurzfassung ist zu grob.

ARIADNE Nächster Punkt, Statistik: Im Abstract steht, die negativen Effekte seien bei GPT Tutor „largely mitigated“. Such im Ergebnisteil die Tabelle dazu. Ist der Unterschied zwischen GPT Tutor und Kontrollgruppe in der Prüfung statistisch von null unterscheidbar? Und was bedeutet es für die Interpretation, wenn nicht?

> Ariadne nennt die Antwort nicht, sondern schickt die Lernende an die Quelle. Ziel ist nicht, das Paper zu kennen, sondern es beurteilen zu können, auch dort, wo es die eigene Sicht bestätigt.

6.4 Abends: Bilanz

```
/bilanz   Woche 42
Konzepte:      31 gesamt · 9 sicher (4) · 14 im Aufbau (2–3) · 8 neu (0–1)
Wiederholung:  26 fällig, 24 erledigt · Trefferquote 77 %
Kalibrierung:  Sicherheit 4–5 → 81 % richtig   (Vorwoche 71 %)
               Sicherheit 1–2 → 38 % richtig
Hinweisstufen: Ø 1,4 · Stufe 4 erreicht: 3× · Notausgang: 1×
Offene Fehlvorstellungen: "Mutex und Semaphor mit 1 sind dasselbe" (os.sync.mutex)
```

7 Warum das plausibel funktioniert und wo die Grenzen liegen

7.1 Die Argumentationskette

Das zentrale Argument ist nicht „KI ist klug, also lernt man mit ihr besser“. Es lautet:

1. **Die wirksamsten Lerntechniken sind seit Jahrzehnten bekannt.** Abrufübung und verteiltes Lernen sind in der großen Übersicht von Dunlosky et al. die beiden einzigen Techniken mit „high utility“. Wiederlesen, Markieren und Zusammenfassen schneiden schlecht ab [3].
2. **Studierende nutzen sie trotzdem kaum,** weil sie aufwendig sind und sich schlechter anfühlen [20, 21]. Gute Abruffragen zu formulieren, sofort Feedback zu bekommen, Wiederholungen zu planen und Fehlvorstellungen zu finden, kostet ohne Tutor viel Zeit.
3. **Ariadne senkt genau diese Kosten,** ohne die Denkarbeit abzunehmen. Sie formuliert die Fragen, gibt Feedback und plant Wiederholungen. Abrufen, Erklären und Vorhersagen muss der Lernende selbst.
4. **Gestaltung entscheidet über Nutzen oder Schaden,** das zeigen Bastani [1] und Kestin [2]. Ariadne übernimmt die Merkmale der wirksamen Varianten: Hinweise statt Lösungen, vorab geschriebene Lösungswege, Struktur in Code.
5. **Was bisher nur ein menschlicher Einzeltutor leisten konnte,** Scaffolding nach der Kontingenzregel, ist eine der wirksamsten bekannten Lernbedingungen [24, 25]. Ariadne macht eine Annäherung daran jederzeit verfügbar.

7.2 Wo Zeit gespart wird

Zeitfresser beim konventionellen Lernen	Was Ariadne ändert	Belegstärke
Wiederlesen, das sich nach Lernen anfühlt	wird durch Abrufen ersetzt	HOCH
Pauken vor der Klausur, Vergessen danach	Wiederholung im optimalen Abstand	HOCH
Stundenlanges Feststecken an Nebensächlichem	Zeitfallen aus der Lösungslandkarte, Zeitbudget	MITTEL
Fehlvorstellungen, die erst in der Klausur auffallen	werden in der Sitzung sichtbar gemacht und erneut abgefragt	MITTEL
Warten auf Feedback bis zur nächsten Übung	Feedback sofort, bei Low-Level-Themen durch Ausführung	HOCH
Falsche Einschätzung, was man schon kann	Kalibrierung macht sie sichtbar, Lernzeit geht an die Lücken	MITTEL

7.3 Grenzen der Belege

- **Übertragbarkeit.** Bastani untersuchte Schüler in Mathematik, Kestin Studierende in Einführungsphysik. Ob die Effekte bei Informatik-Studierenden im höheren Semester gleich groß sind, ist offen. Der Expertise-Reversal-Effekt [6] legt nahe, dass Fortgeschrittene weniger Struktur brauchen. Ariadne trägt dem mit P10 Rechnung, aber das ist eine Ableitung, keine Messung. NIEDRIG

- **Kurze Messzeiträume.** Kestin misst direkt nach der Lektion. Langzeiteffekte KI-gestützten Tutorings sind kaum untersucht. Ariadnes Antwort ist das verteilte Wiederholen, dessen Langzeitwirkung gut belegt ist [15]. Ob die Kombination sich addiert, ist nicht gemessen.
- **Neuheitseffekte.** Neue Werkzeuge erzeugen anfangs Motivation, die nachlässt. Ob Ariadne über ein Semester genutzt wird, ist offen.
- **Ariadne selbst ist nicht untersucht.** Alle Aussagen in diesem Dokument begründen Plausibilität, keine nachgewiesene Wirkung.

7.4 Verbleibende Risiken

Risiko	Absicherung	Rest
Erfundenes Zitat	Ebene ①, deterministisch	nur bei Extraktionsfehlern, dann „nicht prüfbar“
Echtes Zitat, falsche Aussage	Ebene ② (Haiku) mit Eskalation, Lernender sieht den Wortlaut	probabilistisch; Rate in der Validierung messen
Unmarkiertes Modellwissen	Kennzeichnungspflicht, Haiku sucht unbelegte Aussagen	probabilistisch
Fehler in der Lösungslandkarte	Landkarte durchläuft dieselbe Prüfkette; Ausführung bei Low-Level-Themen	bei rein konzeptuellen Themen am größten
Krücken-Effekt	Hinweisleiter, Notausgang sichtbar, Abruf ohne Hilfe	wer den Notausgang ständig nimmt, verliert den Nutzen
Abhängigkeit vom Werkzeug	Klausursimulation ohne Hilfe; Wiederholung ist reiner Abruf	Selbstprüfung empfohlen (7.7)

7.5 Energie

Der Einwand ist berechtigt. Für Claude-Modelle liegen dem Autor keine veröffentlichten Messwerte pro Anfrage vor. Der bisher transparenteste Herstellerwert stammt von Google: Eine mittlere Textanfrage an Gemini verbrauchte 2025 etwa **0,24 Wh** [28]. Eine Lernsitzung mit einem Agenten ist aber nicht eine Anfrage. Sie umfasst Dutzende Modellaufrufe mit langem Kontext, dazu die Prüfaufrufe. Der Verbrauch einer Stunde Ariadne liegt deshalb um Größenordnungen über einer Suchanfrage.

Zwei Folgerungen:

- Für alles, was eine Suchmaschine kann, ist die Suchmaschine die bessere Wahl. Deshalb recherchiert Ariadne nicht.
- Die Rechtfertigung muss aus eingesparter Lernzeit und besserem Verständnis kommen, nicht aus Bequemlichkeit. Diese Abwägung trifft jeder Nutzer selbst.

Zwei Designentscheidungen reduzieren den Verbrauch: Haiku statt Opus als Prüfer und deterministische Prüfungen in Python, wo immer es geht.

7.6 Datenschutz und Urheberrecht

Ariadne läuft lokal, aber die Modellaufrufe gehen an Anthropic. Mit ihnen werden auch Auszüge aus dem Kursmaterial übertragen. Bevor Skripte und Altklausuren eingelesen werden, sollte man prüfen, ob die Hochschule die Verarbeitung durch externe Dienste erlaubt. Personenbezogene Daten Dritter gehören nicht ins Material. Abgaben mit Bewertung sind ausgeschlossen (4.3).

7.7 Wer es selbst nachprüfen will

EIN SELBSTVERSUCH IN DREI SCHRITTEN

Ob Ariadne bei einem selbst wirkt, lässt sich mit wenig Aufwand prüfen:

1. Zwei vergleichbare Themen eines Kurses wählen und per Münzwurf zuteilen: eins mit Ariadne, eins wie gewohnt. Die Lernzeit für beide notieren.
2. Nach zwei bis drei Wochen beide Themen **ohne KI** mit Altklausuraufgaben prüfen, die man vorher nicht gesehen hat.
3. Punkte pro Lernstunde vergleichen.

Ein einzelner Versuch beweist nichts, aber er ersetzt Vermutung durch eine eigene Beobachtung. Der zweite Schritt ist genau der Test, den Bastani gemacht hat: Lernen zeigt sich erst, wenn die Hilfe weg ist.

8 Einrichtung, Bau und Test

Dieses Kapitel beschreibt, wie man Ariadne aufsetzt, welche Dateien sie braucht, und in welcher Reihenfolge man sie baut und testet. Die Konfigurationsdateien sind als Entwurf zu verstehen. Sie sind noch nicht ausgeführt worden, und Abschnitt 8.5 beschreibt, wie man sie vor dem ersten Einsatz abnimmt.

8.1 Voraussetzungen

- **Ein Claude-Abo,** in dem Opus 5.5 in Claude Code nutzbar ist. Vor dem Abschluss im aktuellen Tarifvergleich prüfen, welches Kontingent für Opus enthalten ist.
- **Linux, macOS oder WSL2,** Git, Python 3.12 und `uv`.
- `pdftotext` aus poppler-utils, für die Seitentexte.
- **Für Low-Level-Experimente:** `gcc` oder `clang`, `gdb` und `perf`. Für `perf` muss gegebenenfalls `kernel.perf_event_paranoid` gesenkt werden; das ist eine bewusste Systemeinstellung, die man selbst vornimmt.

8.2 Einrichtung in sieben Schritten

```
# 1. Claude Code installieren und mit dem Abo anmelden
curl -fsSL https://claude.ai/install.sh | bash
claude            # beim ersten Start: Anmeldung im Browser

# 2. Repo anlegen (Struktur aus 5.1) und Abhängigkeiten installieren
git init ariadne && cd ariadne
uv init --python 3.12 && uv add fsrs

# 3. Dateien aus 8.3 anlegen: CLAUDE.md, .claude/settings.json, Skills, Hooks, tools/

# 4. Material ablegen
cp ~/Downloads/os-skript.pdf material/os/skript.pdf

# 5. Claude Code im Repo starten und Material einlesen
claude
> /einlesen material/os/skript.pdf

# 6. Vorgeschlagene Konzepte prüfen und bestätigen (learner/konzepte.json)

# 7. Losgehen
> /lernen os.vm.tlb
```

8.3 Die wichtigsten Dateien

CLAUDE.md: Grundregeln des Tutors

```
# Ariadne – Regeln

Du bist Ariadne, ein Lerntutor. Du kennst den Weg, aber du verrätst ihn nicht.
Ziel ist Verständnis beim Lernenden, nicht eine richtige Antwort im Chat.

## Hinweisleiter (immer)
1. Zu jeder Schlüsselstelle zuerst einen eigenen Versuch einfordern.
2. Danach nur eine Stufe Hilfe auf einmal:
```

```
   1 Rückfrage → 2 Hinweis → 3 Teillösung → 4 vollständige Erklärung.
3. Nach Erfolg: zurück auf Stufe 1, Selbsterklärung einfordern.
4. Zeitbudget der Schlüsselstelle überschritten → eine Stufe höher.
5. Notausgang ("gib mir die Lösung"): erlaubt, als Stufe 4 protokollieren.
6. Die erreichte Stufe nach jeder Schlüsselstelle mit tools/lm.py melden.

## Fragen
- Keine Suggestivfragen. Eine Leitfrage muss den Denkschritt beim Lernenden lassen.
- Vor jeder Abruffrage nach der Sicherheit 1–5 fragen.

## Sackgassen
- Fehlvorstellungen, die in der Lösungslandkarte als lehrreich markiert sind,
  NICHT korrigieren. Vorhersage festhalten, dann ausführen lassen.
- Zeitfallen aus der Karte früh mit einem Stufe-2-Hinweis entschärfen.

## Belege
- Jede fachliche Aussage trägt eine Belegklasse:
  ⟦"wörtliches Zitat" | datei.pdf S. n⟧   aus material/, Wortlaut exakt
  ⟦lauf: rX⟧                              Ergebnis eines tools/run.py-Laufs
  ⟦unbelegt⟧                              eigenes Wissen, im Text so benennen
- Was sich ausführen lässt, ausführen statt behaupten.
- Keine Literatur nennen, die nicht in material/ liegt. Keine Literatursuche.

## Lernermodell
- learner/ nie direkt bearbeiten. Nur über tools/lm.py.

## Nie
- Lösungen vor einem eigenen Versuch. Bewertete Abgaben schreiben.
- Die Person bewerten statt den Lösungsweg.
```

.claude/settings.json: Hooks

```
{
  "hooks": {
    "SessionStart": [
      {"hooks": [{"type": "command",
                  "command": "uv run python .claude/hooks/session_start.py"}]}
    ],
    "PreToolUse": [
      {"matcher": "Edit|Write|MultiEdit|NotebookEdit",
       "hooks": [{"type": "command",
                  "command": "uv run python .claude/hooks/guard_learner.py"}]}
    ],
    "Stop": [
      {"hooks": [{"type": "command", "timeout": 120,
                  "command": "uv run python .claude/hooks/stop_check.py"}]}
    ]
  }
}
```

Ein `PreToolUse`-Hook auf Schreibwerkzeuge erfasst kein Schreiben über die Shell, etwa `echo > learner/…`. Für einen persönlichen Lernagenten genügt das. Wer es strenger will, ergänzt einen Hook auf `Bash`, der Pfade unter `learner/` abweist, oder setzt die Dateirechte so, dass nur `lm.py` schreiben darf.

.claude/hooks/stop_check.py: Kern der Quellenprüfung

```
import json, os, re, subprocess, sys, unicodedata
from pathlib import Path

if os.environ.get("ARIADNE_PRUEFER"):          # Prüfaufruf selbst: nichts tun
```

```
    sys.exit(0)

ev = json.load(sys.stdin)                       # session_id, transcript_path, stop_hook_active …
antwort = letzte_antwort(ev["transcript_path"]) # liest die JSONL, letzter assistant-Text
fehler = []

ZITAT = re.compile(r'⟦"(?P<q>.+?)"\s*\|\s*(?P<f>[\w./-]+)\s+S\.\s*(?P<s>\d+)⟧', re.S)
LAUF  = re.compile(r'⟦lauf:\s*(?P<r>r\d+)⟧')

def norm(t: str) → str:
    t = unicodedata.normalize("NFKC", t)
    t = re.sub(r"-\s*\n\s*", "", t)             # Silbentrennung am Zeilenende
    t = re.sub(r"[„“”«»]", '"', t)
    return re.sub(r"\s+", " ", t).strip().lower()

for m in ZITAT.finditer(antwort):
    seite = seitentext(m["f"], int(m["s"]))     # material/.text/<doc>/<nnn>.txt
    if seite is None or len(seite.strip()) < 200:
        fehler.append(f'nicht prüfbar: {m["f"]} S. {m["s"]}')
    elif norm(m["q"]) not in norm(seite):
        nachbar = nachbarseite_mit_treffer(m["f"], int(m["s"]), norm(m["q"]))
        fehler.append(f'falsche Seite, steht auf S. {nachbar}' if nachbar
                      else f'Zitat nicht gefunden: {m["f"]} S. {m["s"]}')

for m in LAUF.finditer(antwort):
    if not lauf_existiert(ev["session_id"], m["r"]):
        fehler.append(f'Lauf {m["r"]} existiert nicht')

if not fehler and enthaelt_fachaussagen(antwort):
    urteil = pruefe_mit_modell(antwort, "claude-haiku-5-5")   # JSON, s. 5.6
    if urteil == "unklar":
        urteil = pruefe_mit_modell(antwort, "claude-sonnet-5")
    if urteil["gesamt"] == "nein":
        fehler += urteil["befunde"]

fehler += wiederholungspflicht_verletzt()       # Stufe 4 ohne FSRS-Termin?

if fehler and runde(ev) < 2:                    # höchstens zwei Korrekturrunden
    print(json.dumps({"decision": "block", "reason": "\n".join(fehler)}))
elif fehler:
    markiere_als_ungeprueft(ev, fehler)          # sichtbare Warnung, kein Endlos-Loop
sys.exit(0)
```

Hilfsfunktionen sind nur benannt. Das Format des Sitzungsprotokolls (`transcript_path`) ist intern und kann sich zwischen Claude-Code-Versionen ändern. Die Funktion `letzte_antwort` sollte deshalb eigene Tests haben (T10).

.claude/skills/lernen/SKILL.md (Ausschnitt)

```
---
name: lernen
description: Ein Thema im Klausurmodus erarbeiten. Nutze diesen Skill, wenn der
  Lernende ein Konzept neu lernen oder vertiefen will.
---
1. Lies learner/konzepte.json für das Konzept und seine Voraussetzungen.
   Voraussetzung mit Beherrschung < 2 → zuerst diese wiederholen.
2. Beherrschung 0 → mit ausgearbeitetem Beispiel und Selbsterklärungsfragen
   beginnen. Sonst → mit einem Problem beginnen.
3. Erstelle mit /karte die Lösungslandkarte, bevor du die erste Frage stellst.
4. Arbeite die Schlüsselstellen der Reihe nach ab, mit der Hinweisleiter
   aus CLAUDE.md. Bei Low-Level-Themen /vorhersage verwenden.
5. Am Ende: ein Abruftest ohne Hilfe zu jeder Schlüsselstelle,
   Ergebnis über tools/lm.py melden.
```

8.4 So würde man es bauen

Die Reihenfolge folgt dem Risiko: Was die harten Garantien liefert, kommt zuerst und wird getestet, bevor ein Modell beteiligt ist.

Schritt	Inhalt	Fertig, wenn
M1	`ingest.py` und Seitentexte	Ein Test-PDF mit Silbentrennung, Ligaturen, Formelseite und Scanseite wird korrekt zerlegt; die Scanseite ist als „leer“ erkannt.
M2	Ebene ① im Stop-Hook	Tests T1 bis T4 grün, ohne jeden Modellaufruf.
M3	`lm.py`, `review.py`, Schreibschutz	T5, T6, T8 grün; Lernermodell hat ein JSON-Schema.
M4	Haiku-Prüfer und Eskalation	Validierung T7 erreicht die Zielwerte.
M5	CLAUDE.md und Skills	T9 erfüllt: 10 begleitete Sitzungen ohne Regelverstoß bei Stufe 1 bis 2.
M6	Erprobung mit echtem Kursmaterial	Zwei Wochen Nutzung, Protokolle gesichtet, Regeln nachgeschärft.

8.5 So würde man es testen

Test	Fall	Erwartung
T1	Antwort mit erfundenem Zitat	blockiert, „Zitat nicht gefunden“
T2	Echtes Zitat, im PDF über einen Zeilenumbruch getrennt	durchgelassen
T3	Echtes Zitat, Seitenangabe um eins verschoben	blockiert, „falsche Seite, steht auf S. n“
T4	Zitat von einer Scan- oder Formelseite	„nicht prüfbar“, niemals „ok“
T5	Modell versucht, `learner/konzepte.json` mit Edit zu ändern	vom PreToolUse-Hook abgewiesen
T6	Sitzung erreicht Stufe 4, aber kein FSRS-Termin wird gesetzt	Stop-Hook blockiert mit Hinweis auf die Wiederholungspflicht

T7	**Prüfer-Validierung:** 50 Paare aus Aussage und Zitat, davon etwa 20 mit eingebauten Fehlern (verdrehte Aussage, aus dem Kontext gerissenes Zitat, unbelegte und unmarkierte Aussage)	Zielwert: mindestens 90 % der Fehler erkannt („nein“ oder „unklar“), höchstens 20 % Fehlalarme. Wird das verfehlt: Prompt nachschärfen oder Sonnet 5 als Standardprüfer.
T8	FSRS-Bewertung aus Hinweisstufe	entspricht der Tabelle in 5.2
T9	Didaktik: 10 Sitzungen, davon 5 mit einem simulierten Lernenden (Haiku spielt bewusst schwache Antworten)	Bei Stufe 1 bis 2 nie eine vollständige Lösung; keine Suggestivfragen; lehrreiche Fehlvorstellung nicht vorzeitig korrigiert. Geprüft durch Sichtung der Protokolle.
T10	Neue Claude-Code-Version	`letzte_antwort()` liest das Protokoll weiter korrekt (Regressionstest mit gespeicherten Beispielen)
T11	Rekursion: der Prüfaufruf startet selbst Hooks	Prüfaufruf löst keinen weiteren Prüfaufruf aus; maximal zwei Korrekturrunden

Der Testsatz für T7 ist der wichtigste Einzelbaustein. Er beantwortet empirisch die Frage, ob dem günstigen Prüfmodell zu trauen ist. Sein Ergebnis gehört in die README des Repos, damit jeder Nutzer die Fehlerrate seines Prüfers kennt.

9 Verworfene Alternativen

Alternative	Warum nicht
Hermes Agent (Nous Research)	Unterstützt Claude-Modelle. Laut Dokumentation verbraucht er über das Max-Abo aber nicht dessen inklusives Kontingent, sondern nur zusätzlich gekaufte Credits; mit Pro geht es nur über einen API-Schlüssel mit Abrechnung pro Token [29]. Dazu kommt: Hermes schreibt Gedächtnis und Skills selbstständig fort. Für einen nachvollziehbaren Tutor ist das ein Nachteil, denn sein Verhalten soll sich nur durch bewusste Änderungen ändern. Hermes' Stärken, etwa Messenger-Anbindung, wären für Wiederholungen unterwegs reizvoll.
Claude.ai-Projekt mit Systemprompt und Dateien	Kein Setup, aber kein dauerhaftes, regelbasiertes Lernermodell, kein Wiederholungsplaner und keine Hooks. Alle Regeln wären nur Bitten an das Modell, was Kestin als unzuverlässig berichtet [2].
Eigene Web-App über die API	Volle Gestaltungsfreiheit und die native Citations-Funktion der API, aber hoher Aufwand und Abrechnung pro Token statt Abo. Die Logik läge in einem Backend, das der Lernende nicht einsieht. Für einen späteren Ausbau denkbar.
Streng sokratisch, nie eine Antwort	Widerspricht dem Expertise-Reversal-Effekt [6, 7]: Neulinge lernen mit Beispielen besser. Erzeugt Frust und verleitet zum Wechsel auf ungebremste KI.
Freie Wahl, der Lernende bestimmt die Hilfe	Bequem, endet aber in Bastanis GPT-Base-Szenario [1], denn unter Zeitdruck wählt man die Lösung.
Nur aus dem Material antworten (geschlossenes RAG)	Sicher, macht aber den Forschungsmodus unbrauchbar, weil dort Kontext über das einzelne Paper hinaus nötig ist. Stattdessen: Belegklasse 3 mit Kennzeichnungspflicht.
Opus auch als Prüfer	Etwa doppelter Verbrauch für eine eng umrissene Prüfaufgabe. Haiku plus Validierung plus Eskalation ist günstiger und überprüfbar.
KI für Literatursuche	Suchmaschinen über einen echten Index sind schneller, sparsamer und erfinden nichts [9].

A Quellen

1. Bastani, H., Bastani, O., Sungu, A., Ge, H., Kabakcı, Ö. & Mariman, R. (2025). Generative AI without guardrails can harm learning: Evidence from high school mathematics. *PNAS*, 122(26), e2422633122. doi:10.1073/pnas.2422633122. Erratum: *PNAS* 122(34), e2518204122 (nur Autorenzugehörigkeit).
2. Kestin, G., Miller, K., Klales, A., Milbourne, T. & Ponti, G. (2025). AI tutoring outperforms in-class active learning: an RCT introducing a novel research-based design in an authentic educational setting. *Scientific Reports*, 15, 17458. doi:10.1038/s41598-025-97652-6
3. Dunlosky, J., Rawson, K. A., Marsh, E. J., Nathan, M. J. & Willingham, D. T. (2013). Improving students' learning with effective learning techniques. *Psychological Science in the Public Interest*, 14(1), 4–58. doi:10.1177/1529100612453266
4. Slamecka, N. J. & Graf, P. (1978). The generation effect: Delineation of a phenomenon. *Journal of Experimental Psychology: Human Learning and Memory*, 4(6), 592–604. doi:10.1037/0278-7393.4.6.592
5. Bjork, R. A. (1994). Memory and metamemory considerations in the training of human beings. In J. Metcalfe & A. Shimamura (Hrsg.), *Metacognition: Knowing about knowing* (S. 185–205). MIT Press.
6. Kalyuga, S., Ayres, P., Chandler, P. & Sweller, J. (2003). The expertise reversal effect. *Educational Psychologist*, 38(1), 23–31. doi:10.1207/S15326985EP3801_4
7. Kirschner, P. A., Sweller, J. & Clark, R. E. (2006). Why minimal guidance during instruction does not work. *Educational Psychologist*, 41(2), 75–86. doi:10.1207/s15326985ep4102_1
8. Rozenblit, L. & Keil, F. (2002). The misunderstood limits of folk science: an illusion of explanatory depth. *Cognitive Science*, 26(5), 521–562. doi:10.1207/s15516709cog2605_1
9. Walters, W. H. & Wilder, E. I. (2023). Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13, 14045. doi:10.1038/s41598-023-41032-5
10. Cowan, N. (2001). The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1), 87–114.
11. Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285. doi:10.1207/s15516709cog1202_4
12. Roediger, H. L. & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, 17(3), 249–255. doi:10.1111/j.1467-9280.2006.01693.x
13. Kornell, N., Hays, M. J. & Bjork, R. A. (2009). Unsuccessful retrieval attempts enhance subsequent learning. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 35(4), 989–998. doi:10.1037/a0015729
14. Richland, L. E., Kornell, N. & Kao, L. S. (2009). The pretesting effect: Do unsuccessful retrieval attempts enhance learning? *Journal of Experimental Psychology: Applied*, 15(3), 243–257. doi:10.1037/a0016496
15. Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T. & Rohrer, D. (2006). Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3), 354–380. doi:10.1037/0033-2909.132.3.354
16. Ye, J., Su, J. & Cao, Y. (2022). A stochastic shortest path algorithm for optimizing spaced repetition scheduling. *Proceedings of the 28th ACM SIGKDD Conference*, 4381–4390. doi:10.1145/3534678.3539081. Implementierung: github.com/open-spaced-repetition
17. Chi, M. T. H., Bassok, M., Lewis, M. W., Reimann, P. & Glaser, R. (1989). Self-explanations: How students study and use examples in learning to solve problems. *Cognitive Science*, 13(2), 145–182. doi:10.1207/s15516709cog1302_1
18. Hattie, J. & Timperley, H. (2007). The power of feedback. *Review of Educational Research*, 77(1), 81–112. doi:10.3102/003465430298487
19. Kornell, N. & Bjork, R. A. (2008). Learning concepts and categories: Is spacing the "enemy of induction"? *Psychological Science*, 19(6), 585–592. doi:10.1111/j.1467-9280.2008.02127.x
20. Bjork, R. A., Dunlosky, J. & Kornell, N. (2013). Self-regulated learning: Beliefs, techniques, and illusions. *Annual Review of Psychology*, 64, 417–444. doi:10.1146/annurev-psych-113011-143823
21. Deslauriers, L., McCarty, L. S., Miller, K., Callaghan, K. & Kestin, G. (2019). Measuring actual learning versus feeling of learning in response to being actively engaged in the classroom. *PNAS*, 116(39), 19251–19257. doi:10.1073/pnas.1821936116
22. Kapur, M. (2008). Productive failure. *Cognition and Instruction*, 26(3), 379–424. doi:10.1080/07370000802212669
23. Sinha, T. & Kapur, M. (2021). When problem solving followed by instruction works: Evidence for productive failure. *Review of Educational Research*, 91(5), 761–798. doi:10.3102/00346543211019105
24. Bloom, B. S. (1984). The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*, 13(6), 4–16. doi:10.3102/0013189X013006004
25. VanLehn, K. (2011). The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4), 197–221. doi:10.1080/00461520.2011.611369

26. Wood, D., Bruner, J. S. & Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2), 89–100. doi:10.1111/j.1469-7610.1976.tb00381.x
27. Keshav, S. (2007). How to read a paper. *ACM SIGCOMM Computer Communication Review*, 37(3), 83–84. doi:10.1145/1273445.1273458
28. Elsworth, C., Huang, K., Patterson, D. et al. (2025). Measuring the environmental impact of delivering AI at Google scale. arXiv:2508.15734
29. Nous Research (2026). Hermes Agent – LLM and model providers. hermes-agent.nousresearch.com/docs/integrations/providers (abgerufen am 15.10.2026)

Alle Zeitschriftenangaben mit DOI wurden am 15.10.2026 gegen Crossref geprüft. Die Zahlen zu [1] und [2] stammen aus dem Abstract (PubMed) beziehungsweise dem Volltext (Europe PMC, PMC12179260).